



Backtracking y Resolución Sistemática de Problemas

Técnicas de Diseño de Algoritmos II

AGENDA DE LA CLASE

Repaso: Voraces vs Programación Dinámica

Introducción al Backtracking

- Filosofía y principios fundamentales
- Espacio de soluciones y árbol de búsqueda
- Estrategias de poda

Problema de las N-Reinas

- Análisis del problema
- Implementación paso a paso
- Optimizaciones

Otros problemas clásicos

- Sudoku, Laberinto, Subset Sum

Metodología de resolución de problemas

Laboratorio práctico

Tarea para la próxima clase

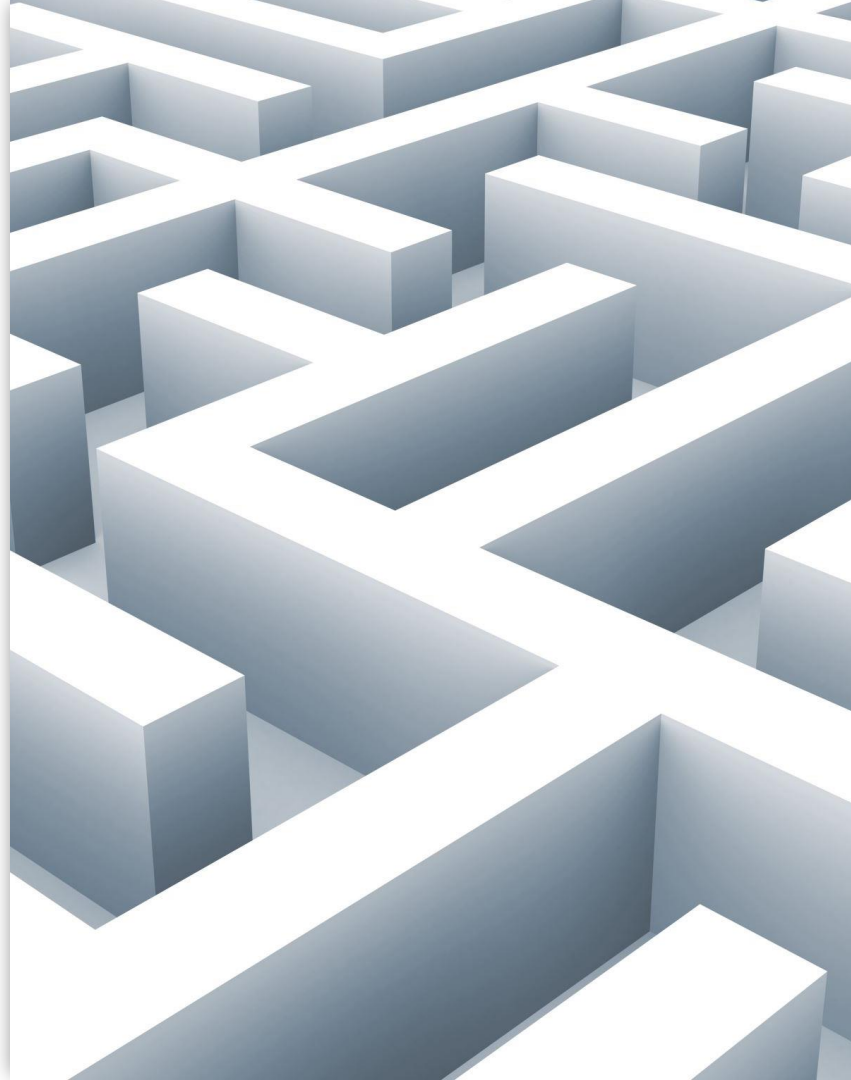
REPASO: TÉCNICAS DE DISEÑO VISTAS

- **Algoritmos Voraces:** Decisión local óptima
 - Ejemplo: Cambio de monedas
 - Ventaja: Eficiencia
 - Desventaja: No siempre funciona
- **Programación Dinámica:** Evalúa todas las opciones
 - Ejemplo: Fibonacci
 - Ventaja: Garantiza optimalidad
 - Desventaja: Puede requerir mucho espacio/tiempo
- **¿Qué pasa cuando necesitamos explorar TODAS las posibilidades?**



INTRODUCCIÓN AL BACKTRACKING

- **Definición:** "Búsqueda sistemática en el espacio de soluciones con retroceso inteligente"
- **Principio fundamental:**
 - Construir solución incrementalmente
 - Si llegamos a un callejón sin salida → retroceder
 - Explorar alternativa diferente
- **Metáfora:** Explorar un laberinto con hilo de Ariadna
- **Cuándo usar:** Problemas de decisión, optimización combinatoria, puzzles



ANATOMÍA DEL BACKTRACKING



Componentes esenciales:

Estado: Representación de solución parcial

Espacio de estados: Todas las configuraciones posibles

Función de validación: ¿Es válida esta configuración?

Función objetivo: ¿Es una solución completa?

Función de poda: ¿Vale la pena continuar?



Árbol de búsqueda: Visualización del proceso



Complejidad: Generalmente exponencial, pero poda la mejora

```
función Backtrack(estado_actual, solución_parcial):  
    si es_solución_completa(solución_parcial):  
        procesar_solución(solución_parcial)  
        return  
  
    para cada opción en generar_opciones(estado_actual):  
        si es_válida(opción, solución_parcial):  
            agregar(opción, solución_parcial)  
            Backtrack(nuevo_estado, solución_parcial)  
            remover(opción, solución_parcial) // BACKTRACK!
```

ESQUEMA GENERAL DEL BACKTRACKING

Clave: El paso de
"deshacer" la
decisión

PROBLEMA DE LAS N-REINAS - ENUNCIADO

- **Planteamiento:** Colocar N reinas en un tablero $N \times N$ de manera que ninguna pueda atacar a otra **Reglas del ajedrez:** Las reinas atacan horizontal, vertical y diagonalmente **Historia:** Planteado por Max Bezzel en 1848 para $N=8$ **Aplicaciones modernas:** Asignación de recursos sin conflictos
- Planificación de horarios
- Colocación de antenas celulares
- **¿Cuántas soluciones existen para $N=8$?**



N-REINAS - ANÁLISIS DEL ESPACIO DE SOLUCIONES



Enfoque ingenuo: $C(64,8) \approx 4.4 \times 10^9$ combinaciones
Optimización 1: Una reina por fila $\rightarrow 8^8 \approx 16.7$ millones
Optimización 2: Una reina por fila y columna $\rightarrow 8! = 40,320$
Representación eficiente: Array unidimensional

`queens[i] = j` significa: reina en fila i , columna j



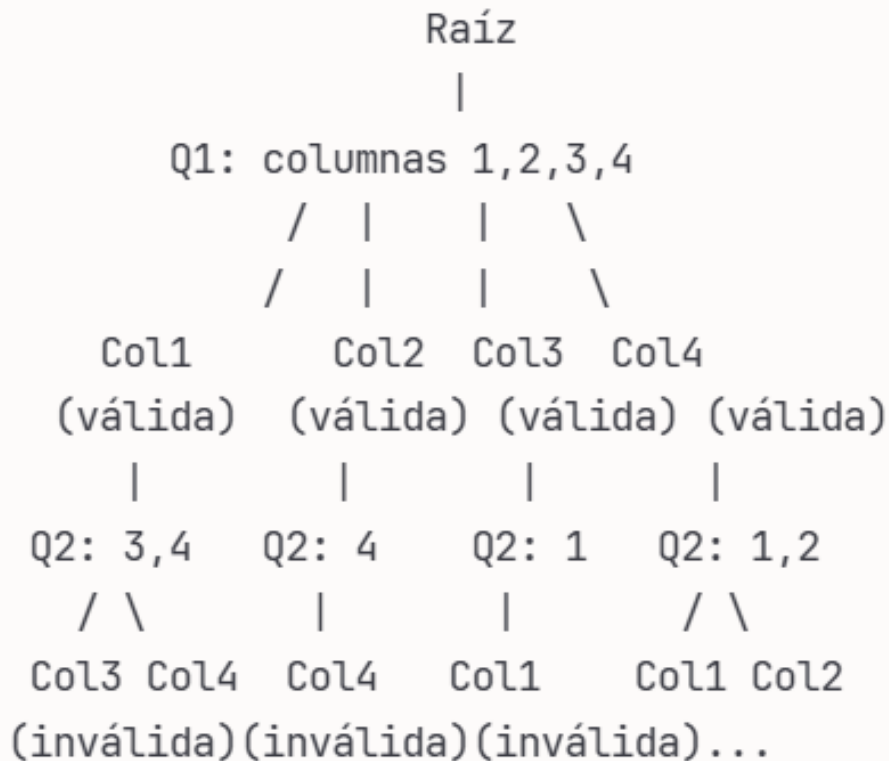
Conflictos a verificar:

Columnas: `queens[i] != queens[k]`

Diagonales: $|i-k| \neq |queens[i] - queens[k]|$

ÁRBOL DE BÚSQUEDA - N-REINAS (N=4)

- **Poda temprana:**
Eliminar ramas que no pueden llevar a solución
- **Visualización interactiva:** [Enlace a simulador]



IMPLEMENTACIÓN N-REINAS

python

```
def n_reinas(n):
    def es_seguro(tablero, fila, col):
        for i in range(fila):
            # Verificar columna y diagonales
            if (tablero[i] == col or
                abs(tablero[i] - col) == abs(i - fila)):
                return False
        return True

    def backtrack(tablero, fila):
        if fila == n:
            soluciones.append(tablero[:])
            return

        for col in range(n):
            if es_seguro(tablero, fila, col):
                tablero[fila] = col
                backtrack(tablero, fila + 1)
                # Backtrack implícito (se sobrescribe)

    soluciones = []
    backtrack([0] * n, 0)
    return soluciones
```

OPTIMIZACIONES EN N-REINAS

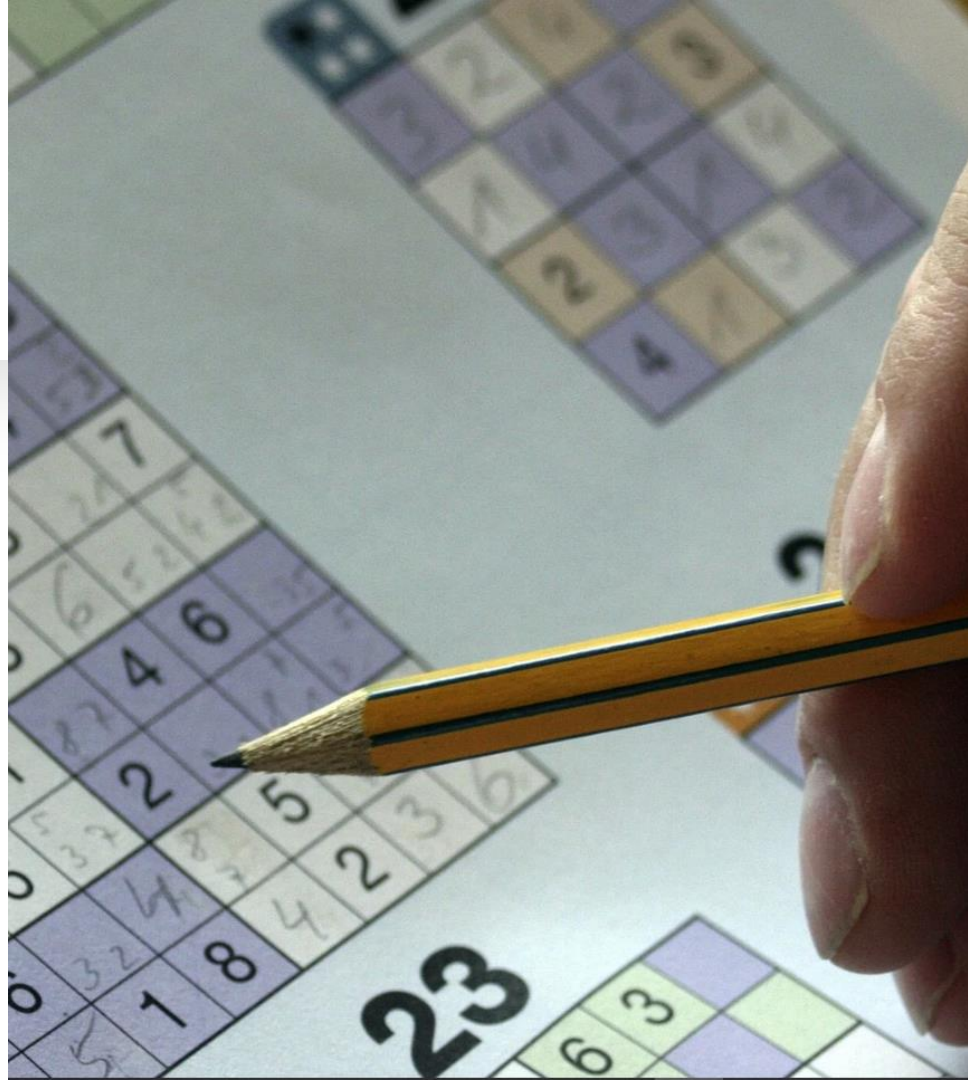
- **Sets para validación $O(1)$:** `cols_ocupadas = set()`
 - `diag_pos = set()` # fila + columna
 - `diag_neg = set()` # fila - columna
- **Simetría:** Aprovechar rotaciones y reflexiones **Heurísticas de ordenamiento:**
 - Probar primero columnas centrales
 - Most Constrained Variable (MCV)
- **Paralelización:** Dividir por primera fila
- **Resultado:** $N=8 \rightarrow 92$ soluciones únicas, 12 fundamentales

OTROS PROBLEMAS CLÁSICOS DE BACKTRACKING

Sudoku (9x9): Estado: Tablero parcialmente lleno	Restricciones: Filas, columnas, subcuadrados Heurística: Llenar celdas con menos opciones primero
Problema de la mochila (versión decisión):	¿Existe subconjunto con suma exacta S ?
Coloreo de grafos:	Asignar colores a vértices sin adyacentes iguales
Encontrar camino en laberinto:	Explorar todas las direcciones posibles

SUDOKU - CASO DE ESTUDIO

- **Complejidad:** $O(9^{(celdas_vacías)})$ en el peor caso
- **Técnicas de poda efectivas:**
 - Constraint propagation
 - Naked singles, hidden singles
 - Arc consistency
- **Implementación híbrida:** Backtracking + heurísticas
- **Datos curiosos:**
 - Mínimo de 17 pistas para solución única
 - Puzzles más difíciles no necesariamente tienen menos pistas
- **Aplicación en IA:** CSP (Constraint Satisfaction Problems)



METODOLOGÍA DE RESOLUCIÓN DE PROBLEMAS

Entender el problema

- Leer cuidadosamente el enunciado
- Identificar entrada, salida y restricciones
- Hacer ejemplos pequeños manualmente

Identificar la técnica apropiada

- ¿Es optimización? → DP o Voraz
- ¿Es búsqueda exhaustiva? → Backtracking
- ¿Se puede dividir? → Divide y vencerás

Diseñar la solución

- Definir estados y transiciones
- Identificar caso base y recursivo
- Pensar en optimizaciones

Implementar y verificar

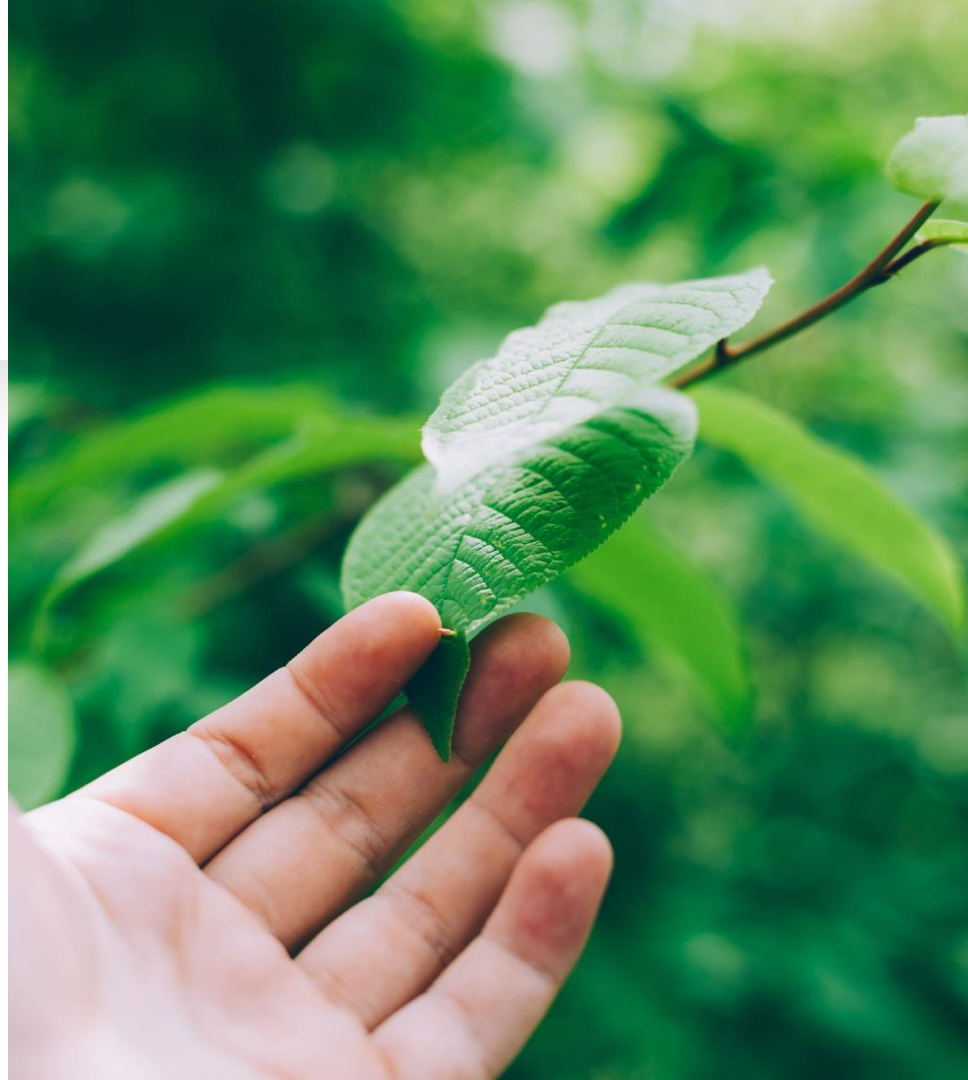
- Casos de prueba pequeños
- Validar complejidad esperada

BACKTRACKING vs OTRAS TÉCNICAS

Aspecto	Backtracking	Programación Dinámica	Algoritmos Voraces
Propósito	Búsqueda exhaustiva	Optimización	Optimización rápida
Garantía	Encuentra todas/mejor	Solución óptima	No garantiza óptimo
Complejidad	Exponencial (con poda)	Polinómica típica	Polinómica baja
Espacio	$O(\text{profundidad})$	$O(\text{subproblemas})$	$O(1)$ típicamente
Cuándo usar	Espacios finitos	Subproblemas superpuestos	Propiedades voraces
Ejemplos	N-reinas, Sudoku	Fibonacci, LCS	Dijkstra, Huffman

ESTRATEGIAS DE PODA AVANZADAS

- **Poda por cotas (Branch and Bound):**
 - Mantener mejor solución conocida
 - Descartar ramas que no pueden mejorarla
- **Poda por dominancia:**
 - Si estado A es mejor que B en todos los aspectos
- **Ordenamiento inteligente:**
 - Explorar opciones más prometedoras primero
 - Fail-fast: probar restricciones más estrictas
- **Memoización en backtracking:**
 - Recordar estados ya explorados (cuando es posible)



HERRAMIENTAS Y VISUALIZACIÓN

Debugging de backtracking:

- Imprimir árbol de recursión
- Contar nodos explorados vs podados
- Visualizar progreso con indentación

Herramientas online:

- VisuAlgo: visualizaciones interactivas
- Algorithm Visualizer
- Recursive Backtracking Simulator

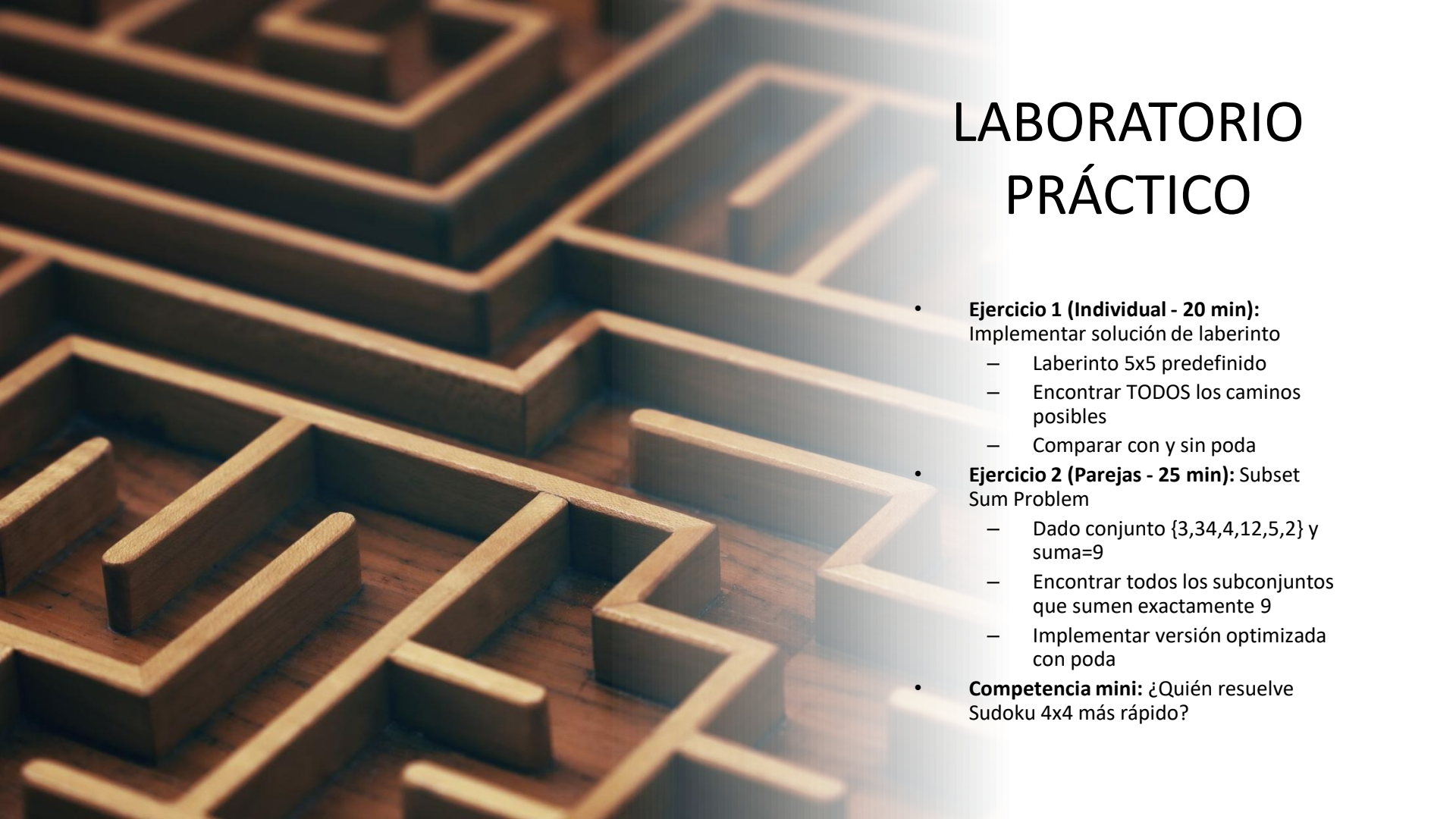
Profiling específico:

- Medir efectividad de poda
- Analizar distribución de soluciones en árbol

APLICACIONES EN LA INDUSTRIA

- **Google:** Asignación de servidores y recursos
 - Problema de empaquetado multidimensional
- **Spotify:** Creación de playlists personalizadas
 - Optimización con restricciones de diversidad
- **Airlines:** Scheduling de tripulaciones
 - Restricciones laborales y regulatorias complejas
- **Gaming:** IA para juegos de estrategia
 - Chess engines, pathfinding con obstáculos dinámicos
- **Logística:** Ruteo de vehículos con restricciones
 - Amazon, MercadoLibre, Rappi





LABORATORIO PRÁCTICO

- **Ejercicio 1 (Individual - 20 min):** Implementar solución de laberinto
 - Laberinto 5x5 predefinido
 - Encontrar TODOS los caminos posibles
 - Comparar con y sin poda
- **Ejercicio 2 (Parejas - 25 min):** Subset Sum Problem
 - Dado conjunto $\{3, 34, 4, 12, 5, 2\}$ y suma=9
 - Encontrar todos los subconjuntos que sumen exactamente 9
 - Implementar versión optimizada con poda
- **Competencia mini:** ¿Quién resuelve Sudoku 4x4 más rápido?

CASOS DE ESTUDIO REGIONALES

- **Ualá (Fintech argentina):** Detección de patrones de fraude
 - Backtracking para encontrar secuencias sospechosas
- **Mercado Libre:** Optimización de almacenes
 - Ubicación óptima de productos con restricciones físicas
- **Telecom Argentina:** Planificación de redes
 - Colocación de antenas evitando interferencias
- **Despegar:** Armado de paquetes turísticos
 - Combinaciones vuelo+hotel+auto con restricciones temporales



LIMITACIONES Y CONSIDERACIONES

Cuándo NO usar backtracking:

- Espacios de búsqueda continuos
- Cuando existe algoritmo polinómico conocido
- Problemas con demasiada simetría

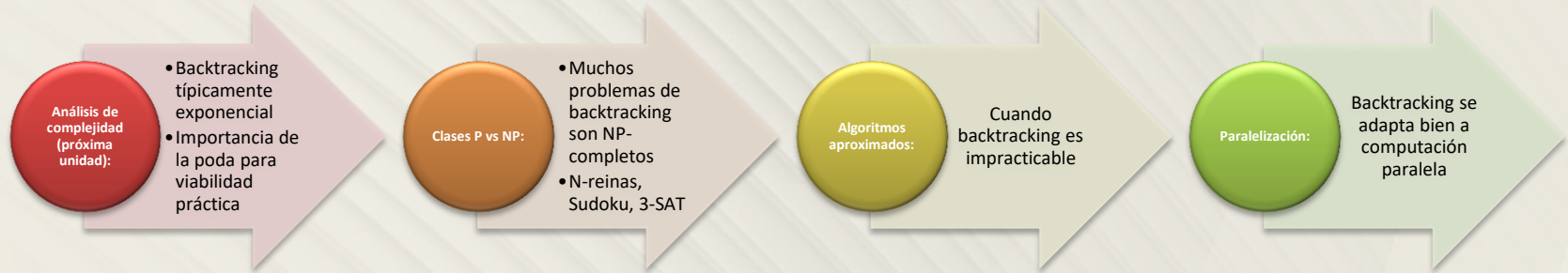
Mejores alternativas:

- Algoritmos aproximados para problemas NP-hard
- Heurísticas metaheurísticas (Genetic Algorithms, Simulated Annealing)
- Constraint satisfaction solvers (CP-SAT)

Trade-offs:

- Exactitud vs Tiempo de ejecución
- Memoria vs Velocidad (iterativo vs recursivo)

CONEXIÓN CON PRÓXIMOS TEMAS



TAREA PARA LA PRÓXIMA CLASE

Implementaciones obligatorias:

- N-reinas en Python (con al menos 2 optimizaciones)
- Sudoku solver básico en Haskell
- Subset sum con análisis de poda

Lectura preparatoria:

- Cormen et al., Capítulo 28 (Backtracking algorithms)
- Kleinberg & Tardos, Capítulo 5 (Complete search)

Análisis experimental:

- Medir tiempo N-reinas para $N=4,6,8,10$
- Graficar efectividad de diferentes podas

Entrega: Campus virtual, deadline domingo 23:59

PRÓXIMA CLASE

Tema: Medición de Rendimiento y Casos Prácticos (Unidad 2)

Contenidos principales:

- Análisis asintótico: mejor caso, promedio, peor caso
- Herramientas de profiling y benchmarking
- Comparación empírica de algoritmos
- Casos de estudio con algoritmos reales

Preparar: Laptop con herramientas de profiling instaladas

Modalidad: Laboratorio intensivo con mediciones